

RAPPORT DE REMÉDIATION TECHNIQUE

Un système qui n'était pas fragile. Il était ouvert.

Plateforme RH HRFlow – remédiation conduite à la suite de l'incident P1 du 14 août 2024 et de l'audit Partech du 18 septembre 2024.

51	12	279	5	1,8 M€
constats d'audit	critiques, tous fermés	tests automatisés	étapes de pipeline	conditionnés à ce travail

Commanditaire	NovaTech SAS – Karim Bouaziz, directeur général
Destinataire	Partech Ventures – contre-audit du second versement
Périmètre	Dépôt novatech-hrflow – 5 services, 1 frontend, chaîne de livraison, infrastructure
Bloc de compétences	BC03 – RNCP 38822, niveau 7
Dépôt	github.com/lukunkuSarah/novatech-hrflow

Synthèse

L'audit technique commandé par Partech décrivait une dette accumulée depuis 2021. Notre propre audit, conduit sans reprendre la liste existante, aboutit à un constat plus direct : **HRFlow n'était pas un système fragile, c'était un système ouvert.**

À la date de la reprise, un attaquant sans compte, sans outil et sans connaissance préalable pouvait, en moins de dix minutes, lire l'intégralité des données RH de tous les clients, prendre le contrôle de n'importe quel compte, provoquer une coupure totale de la plateforme et forger des jetons d'authentification valides. Ces quatre expositions ne relevaient pas de la dette technique : c'étaient des vulnérabilités actives.

Ce rapport documente 51 constats, dont 12 que l'audit externe n'avait pas relevés. Trois d'entre eux invalident des correctifs que l'on serait tenté d'appliquer en confiance : réactiver les tests, s'appuyer sur le build existant, déployer d'abord en environnement de recette.

Résultat en quatre chiffres

INDICATEUR	14 AOÛT 2024	OBJECTIF PARTECH	AUJOURD ' HUI
Détection d'un incident P1	2 h 28	< 2 min	≈ 90 s
Rétablissement du service	3 h 07	< 10 min	≈ 2 min
Perte de données maximale	1 h 17	< 15 min	≤ 15 min
Couverture de tests	0 %	≥ 80 %	95-100 %
Vulnérabilités critiques ouvertes	12	0	0
Vulnérabilités de dépendances	6	0	0

Un point reste ouvert et nous le signalons sans le minimiser : **la rotation effective des secrets de production** exige des accès dont une équipe de développement ne dispose pas. La procédure est écrite ; son exécution appartient à la direction technique.

1 · L'audit : méthode et résultats

1.1 Une relecture menée à froid

L'audit a été conduit en quatre passes volontairement séparées. La deuxième est la seule qui compte vraiment : **100 % des fichiers relus ligne à ligne, sans la liste Partech sous les yeux**. C'est elle qui a produit les douze constats supplémentaires. Se contenter de confirmer une liste existante aurait donné un rapport plus rapide et strictement inutile.

Les commentaires laissés dans le code — `// TODO`, `// à retirer`, `// jamais fait` — ont été traités comme des indices, jamais comme des preuves. Les problèmes les plus graves ne portaient aucun commentaire : personne, dans l'équipe précédente, n'en avait conscience.

1.2 Répartition des 51 constats

GRAVITÉ	NOMBRE	DÉFINITION RETENUE
Critique	12	exploitable immédiatement, sans authentification, impact majeur
Élevé	21	exploitable sous condition, ou impact structurel
Moyen	17	défaut avéré sans exploitation directe
Faible	1	gêne opérationnelle

1.3 Les douze constats absents de l'audit externe

ID	CONSTAT	POURQUOI IL COMPTE
QUA-07	<code>npm test</code> exécutait <code>echo && exit 0</code>	Réactiver les tests dans le pipeline n'aurait rien changé : la barrière serait restée verte quoi qu'il arrive.
QUA-09	Le frontend n'était pas constructible	L'étape « build » ne pouvait pas réussir. Elle était décorative.
INF-05	<code>deploy.sh</code> staging déployait la production	Le script lisait l'argument d'environnement et ne s'en servait jamais.
SEC-03	Prise de contrôle de compte sans jeton	Plus grave que l'injection SQL : ne demande aucune compétence technique.
SEC-07	Téléversement non contrôlé, traversée de répertoire	Dépôt de fichier arbitraire sur le serveur de production.
SEC-08	Aucun cloisonnement multi-locataire	Défaut d'architecture, invisible ligne par ligne.
SEC-13	Aucune limitation de débit sur l'authentification	Force brute triviale, même après correction de l'injection SQL.
QUA-01	Aucun <code>try/catch</code> sur les routes asynchrones	Déni de service par une requête unique, sur les cinq services.
QUA-02	Calcul de paie erroné	Le risque n'est pas l'absence de test : ce sont les bulletins déjà émis.
QUA-03	Virements sans clé d'idempotence	Un rejeu paie deux fois le même salarié.
QUA-05	Solde de congés augmentable par dates inversées	Fraude exploitable par tout salarié, sans outil.
SEC-18/19/21	Chaîne d'authentification côté navigateur	Reste cassée même après correction du backend.

1.4 Deux exemples représentatifs

La fraude aux congés — une ligne, trois défauts

```
const nombreJours = Math.ceil(  
  (new Date(dateFin) - new Date(dateDebut)) / (1000 * 60 * 60 * 24)  
)
```

Des dates inversées produisent une valeur **négative**, qui vient *augmenter* le solde du salarié. Les week-ends et jours fériés sont décomptés comme ouvrés. La borne finale est exclue, donc un congé d'une journée vaut zéro jour. Corrigé à trois niveaux : domaine, application, et contrainte de base — CHECK (`date_fin >= date_debut`), parce qu'une contrainte de base ne se commente pas « temporairement ».

La prise de contrôle de compte — le défaut était une absence

```
app.post('/auth/reset-password', async (req, res) => {
  const { email } = req.body
  // aucun jeton, aucune vérification de propriété
  const newPassword = Math.random().toString(36).slice(-8)
  await pool.query('UPDATE users SET password_hash = $1 WHERE email = $2', [hash, email])
  console.log(`[RESET] New password for ${email}: ${newPassword}`)
})
```

La route réinitialisait le mot de passe de **n'importe quelle adresse fournie**, sans preuve de possession, et l'écrivait en clair dans un journal que Nginx servait publiquement sur `/logs/`. Deux vulnérabilités qui se composent : la seconde transformait la première en compromission silencieuse.

2 · Les décisions structurantes

Huit décisions sont consignées dans `docs/ADR.md` avec leur contexte et leurs conséquences, y compris négatives. Les cinq qui engagent le plus :

ADR-001 – Supprimer les routes dangereuses plutôt que les sécuriser

L'audit externe recommandait de « sécuriser ou supprimer » la route de migration. Nous avons supprimé. Une migration de schéma n'est pas une fonctionnalité de l'application : elle ne doit pas s'exécuter dans le processus qui sert le trafic, quel que soit le contrôle d'accès exigé pour la déclencher. Une route de debug protégée reste une route de debug — sa protection est ce qu'on oublie de vérifier.

ADR-002 – Refuser de démarrer plutôt qu'appliquer une valeur par défaut

Le code contenait `process.env.JWT_SECRET || 'novatech_jwt_super_secret_key_...'`. Ces valeurs de repli permettent à un service de démarrer dans un environnement mal configuré. C'est précisément le problème : il démarre, il paraît fonctionner, et il signe des jetons avec un secret public. Un service dont la configuration est incomplète refuse désormais de démarrer.

ADR-004 – Aucune action tierce dans le pipeline

Le pipeline audité utilisait `appleboy/ssh-action@master` : une référence *mutable*, dont le code s'exécutait automatiquement avec les identifiants de production. Le fichier de workflow est plus verbeux aujourd'hui — le déploiement SSH est écrit à la main — mais il est auditable.

ADR-005 – Reprendre le barème de paie à l'identique

Les taux de cotisation étaient annotés « approximatif » et « à vérifier avec le comptable ». Nous avons corrigé ce qui relève de la technique — arrondis au centime en arithmétique entière, idempotence, transactions — et repris les taux **sans y toucher**. Corriger un taux réglementaire sans validation d'un expert-comptable remplacerait une erreur par une autre, avec la même apparence de justesse.

ADR-007 – Des drapeaux en configuration, pas un service externe

Le temps partiel n'est pas couvert par le barème : plutôt que d'émettre un bulletin faux, le service refuse. La fonctionnalité est déployée mais éteinte, et s'activera entreprise par entreprise une fois le barème validé. Un bulletin non émis est un incident visible ; un bulletin faux est un contentieux différé.

3 · Métriques et preuves

3.1 Couverture de tests

PÉRIMÈTRE	TESTS	INSTRUCTIONS	BRANCHES	FONCTIONS	LIGNES
shared	93	95,4 %	80,0 %	97,4 %	97,2 %
api-gateway	23	100 %	82,4 %	100 %	100 %
auth	34	99,0 %	100 %	93,8 %	99,0 %
paie	47	98,3 %	95,9 %	100 %	100 %
conges	38	99,1 %	89,5 %	93,3 %	99,0 %
recrutement	33	96,0 %	83,3 %	100 %	95,7 %
frontend	3 + 8 E2E	parcours navigateur, Chromium			

Le seuil de 80 % est **bloquant** et déclaré par service. Une part importante de ces tests ne vérifie pas une fonctionnalité mais **l'absence d'une vulnérabilité** : que /paie/migrate réponde 404, qu'un jeton alg: none soit refusé, qu'aucun secret ne puisse être journalisé.

Un test a détecté un défaut réel introduit pendant la remédiation. La passerelle retirait le préfixe de montage et transmettait /solde/10 au lieu de /conges/solde/10. Le test d'intégration de la passerelle l'a signalé avant toute mise en service. C'est la démonstration la plus concrète de ce qu'apporte la barrière.

3.2 Chaîne de livraison

ÉTAPE	CONTENU	CE QU'ELLE REND IMPOSSIBLE
1 · Build	npm ci, lint, format, image immuable taguée par empreinte de commit	Déployer autre chose que ce qui a été testé
2 · Test	279 tests, couverture ≥ 80 %, 8 parcours E2E	Rejouer la régression « heures supplémentaires » du commit 30e906c
3 · Security	gitleaks, npm audit, Trivy, 14 contrôles de motifs interdits	Recommiter un .env de production
4 · Staging	environnement isolé, migrations hors trafic, tests de fumée	Un « staging » qui déploie la production
5 · Production	approbation humaine, bascule sans interruption, retour arrière automatique	Le scénario du 14 août

3.3 Supervision — les quatre signaux d'or

Le système audité n'exposait aucune métrique. Sa seule sonde répondait 200 OK en toutes circonstances : une supervision branchée dessus serait restée verte pendant les 3 h 07 de l'incident.

SIGNAL	MÉTRIQUE	SEUIL D'ALERTE
Latence	hrflow_duree_requete_secondes — centiles 50/95/99	p95 > 2 s pendant 5 min
Trafic	hrflow_requetes_total par service et par route	—
Erreurs	part des réponses 5xx	> 1 %
Saturation	mémoire, CPU, retard de boucle d'événements	mémoire > 192 Mo sur 256

Un histogramme, pas une moyenne. Une réponse sur cent à trois secondes ne déplace pas la moyenne, mais c'est celle que l'utilisateur remarque. **Les routes sont normalisées en gabarits** : sans quoi 8 200 salariés produiraient 8 200 séries temporelles.

4 · Post-mortem de l'incident P1 du 14 août 2024

RELECTURE PAR L'ÉQUIPE ENTRANTE – COMPLÈTE LE POST-MORTEM RÉDIGÉ LE 16 AOÛT PAR LA DIRECTION GÉNÉRALE

4.1 Chronologie

HEURE	ÉVÉNEMENT	CE QUE LE SYSTÈME AURAIT DÛ FAIRE
23:30	Migration SQL lancée en production via POST /paie/migrate	Refuser : la route ne devrait pas exister
23:47	La table employees est corrompue, la plateforme tombe	Alerter dans les 90 secondes
– 2 h 28 –	Aucune alerte. Aucun tableau de bord. La sonde répondait 200.	–
02:15	Un client d'hôtel appelle le numéro d'urgence	L'astreinte aurait dû être en action depuis 2 h 26
02:25	Tentative de retour arrière – aucune procédure documentée	Retour arrière scripté, ≈ 2 min
02:54	Restauration d'une sauvegarde de 22 h 30 – 1 h 17 perdues	Sauvegarde horaire vérifiée, ≤ 15 min

4.2 Causes racines – notre lecture

Le post-mortem d'origine identifie cinq causes techniques. Elles sont exactes, mais elles décrivent des symptômes. La cause commune est ailleurs.

Aucun de ces problèmes n'est apparu par surprise. Tous étaient connus, documentés dans le code *par leurs propres auteurs*, et repoussés. Le middleware d'authentification portait // TODO: remettre après fix depuis mars 2024. La route de debug portait // à retirer depuis juillet. Le .gitignore portait // TODO : à corriger – Théo 14/08/2024 (jamais fait).

La défaillance n'est pas technique, elle est **processuelle** : rien, dans la chaîne de livraison, n'empêchait de livrer du code défectueux.

4.3 Ce que le post-mortem d'origine ne dit pas

Il liste cinq actions décidées. Dix jours plus tard, son auteur écrivait : « *Status : aucune action réalisée à ce jour.* » Aucune de ces cinq actions ne portait de responsable ni d'échéance.

Une action corrective sans responsable ni date n'est pas une action corrective. C'est pourquoi chaque ligne de notre suivi de remédiation renvoie à une preuve exécutable – un test qui échoue si la vulnérabilité se rouvre, ou une vérification que le pipeline applique à chaque demande de fusion.

4.4 Le même scénario, aujourd'hui

ÉTAPE DU 14 AOÛT	AUJOURD'HUI
Appel de POST /paie/migrate	404 – la route n'existe plus, et deux tests le vérifient
Migration exécutée dans le processus de production	Conteneur éphémère, hors trafic, après sauvegarde vérifiée
UPDATE employees sans clause WHERE	Supprimé de la migration – il n'apportait rien
Panne non détectée pendant 2 h 28	Alerte à l'astreinte en $\approx$ 90 s
Retour arrière improvisé	scripts/rollback.sh, images déjà présentes, $\approx$ 2 min
Sauvegarde de 1 h 17	Sauvegarde avant chaque déploiement, plus horaire, intégrité vérifiée

5 · Ce qui reste ouvert

Un rapport crédible énonce aussi ses limites. Ces points ne sont pas des oublis : ce sont des décisions qui n'appartiennent pas à l'équipe technique, ou des actes qui exigent des accès dont un dépôt de code ne dispose pas.

SUJET	NATURE	DÉCIDEUR
Rotation effective des secrets de production	opérationnel — exige les accès de production	direction technique
Validation du barème de cotisations	comptable et juridique	expert-comptable
Sort des bulletins déjà émis avec un calcul erroné	juridique	direction générale
Protection de branche, relecteurs obligatoires	configuration de la forge	administrateur du dépôt
Certificats TLS, empreintes d'hôtes	opérationnel	administrateur système
Infrastructure cloud provisionnée	exige un compte et un budget	direction technique

Sur l'historique Git

Les secrets d'origine étaient présents dans les premiers commits. Lors de la publication du dépôt, **la protection de secrets de GitHub a refusé le push** en détectant la clé Stripe — confirmation indépendante du constat SEC-01. L'historique a été purgé conformément à notre propre plan : le `.env` est retiré des 37 commits et les onze valeurs de secrets sont remplacées, sans effacer le code vulnérable d'origine, qui reste consultable.

Cela ne dispense pas de la révocation. Un secret publié est un secret compromis, même retiré : la purge traite la trace, pas l'exposition passée.

Executive summary – English

Context. NovaTech's HRFlow is a SaaS HR platform serving 8,200 users across 47 client companies, under a 99.5 % monthly availability commitment. A three-hour outage on 14 August 2024, followed by a technical audit commissioned by Partech Ventures, froze €1.8 M of a funding round pending a credible remediation plan.

What we found. Our own audit – deliberately conducted without the external report at hand – recorded **51 findings, 12 of which the external audit had missed**. The system was not fragile; it was open. An unauthenticated attacker could read every client's HR data, take over any account, bring the platform down, and forge valid authentication tokens.

Three of the twelve additional findings invalidate remediations one would otherwise apply in good faith: the root `npm test` script returned success unconditionally, so re-enabling the test gate would have changed nothing; the frontend could not be built, so the pipeline's build stage was decorative; and the deployment script accepted an environment argument it never used, meaning `deploy.sh staging` deployed production.

Outcome

METRIC	14 AUG 2024	TARGET	NOW
Time to detect (MTTD)	2 h 28	< 2 min	≈ 90 s
Time to recover (MTTR)	3 h 07	< 10 min	≈ 2 min
Recovery point (RPO)	1 h 17	< 15 min	≤ 15 min
Test coverage	0 %	≥ 80 %	95–100 %
Open critical vulnerabilities	12	0	0

How it was done

- **A five-stage pipeline** – build, test, security, staging, production – where four stages block the merge. Each gate closes an incident that has *already happened*; none is theoretical.
- **279 automated tests**, a large share of which assert the *absence* of a vulnerability rather than the presence of a feature.
- **Immutable container images** tagged by commit hash. The server no longer runs `git pull` and rebuilds: what is tested is exactly what runs.
- **Honest health probes**. The previous endpoint returned `200 OK` unconditionally – monitoring wired to it would have stayed green throughout the outage.
- **Scripted, tested rollback** in roughly two minutes. The August outage lasted three hours not because of the migration, but because nobody knew how to undo it.

What we deliberately did not do

We did not correct the payroll contribution rates. They are wrong, and we say so — but changing a statutory rate without a chartered accountant's validation would replace one error with another, wearing the same appearance of correctness. Part-time payslips are therefore *refused* rather than computed: an unissued payslip is a visible incident, an incorrect one is a deferred dispute.

We did not migrate to Kubernetes. Five services, 8,200 users, and a team that has just lost its only expert: the added operational complexity would outweigh the benefit. We adopted the model matching actual maturity, not the ideal one.

Secret rotation remains outstanding. The procedure is written; execution requires production credentials a development team does not hold. Notably, GitHub's own push protection *rejected* our first push after detecting the Stripe key in history — an independent confirmation of finding SEC-01.

The core insight. None of these problems appeared by surprise. Every one was known, documented in the code by its own authors, and postponed. The failure was not technical but procedural: nothing in the delivery chain prevented defective code from shipping. The pipeline does not make the team more disciplined — it makes indiscipline visible immediately, rather than in an audit report six months later.

Annexe – Où trouver quoi

DOCUMENT	CONTENU
docs/00-AUDIT-J1.md	Rapport d'audit – 51 constats tracés, priorisés, localisés au fichier et à la ligne
docs/01-REMEDIATION.md	Suivi – la preuve exécutable de chaque correctif, constat par constat
docs/PLAN-DE-TESTS.md	Stratégie de test, matrice de couverture, traçabilité constat → test
docs/architecture.md	Architecture, modèle de données, choix assumés
docs/RUNBOOK.md	Procédures d'incident, retour arrière, restauration, obligations RGPD
docs/MONITORING.md	Supervision, quatre signaux d'or, routage des alertes, astreinte
docs/ADR.md	Huit décisions d'architecture, avec leurs coûts
docs/openapi.yaml	Contrats d'interface des 16 routes
docs/soutenance/index.html	Support de soutenance – 16 compositions
.github/workflows/pipeline.yml	Pipeline en cinq étapes
monitoring/	Prometheus, Alertmanager, tableau de bord Grafana provisionné

RAPPORT DE REMÉDIATION – ÉQUIPE BB_NUMERIQUE – NOVATECH HRFLOW – DOCUMENT INTERNE